

컴퓨터 구조론

HW2 Report



32217072 모바일시스템공학과 김도익
컴퓨터 구조론 HW2

목차

1. 서론

2. 본론

1) 배경지식

- datapath란
- control logic이란
- single cycle이란

2) 디자인

3) 코드설명

4) 구현

5) 프로그램 실행

3. 결론

1. 서론

이 레포트는 MIPS를 구현한 코드를 바탕으로 Datapath의 작동 방식과 각 명령어의 실행 과정을 설명한다. 구현된 코드는 MIPS명령어를 디코딩하고 실행하며, 이를 통해 레지스터 및 메모리를 업데이트한다. 각 명령어의 실행과정과 그에 따른 Datapath의 변화를 중심으로 설명한다.

2. 본론

1) 배경지식

● Datapath란

Datapath는 프로세서 내에서 데이터가 이동하는 경로와 데이터 처리에 필요한 구성 요소들이다. 데이터 경로는 주로 연산을 수행하는 ALU, 레지스터 파일, 명령어와 데이터를 저장하는 메모리로 구성된다.

Datapath의 동작

데이터 경로의 동작은 명령어 사이클에 따라 이루어진다. 사이클은 다음과 같은 단계로 구성된다.

1: Instruction Fetch:

- Pc가 가리키는 주소에서 명령어를 가져온다.
- 가져온 명령어는 명령어 레지스터(IR)에 저장된다.

2: Instruction Decode:

- 명령어를 해독하여 연산에 필요한 operand와 제어 신호를 추출한다.

- 레지스터 파일에서 operand를 읽어온다.

3: Execution:

- ALU를 사용하여 연산을 수행한다. 연산의 종류는 명령어에 따라 달라진다.

4: Memory Access

- load명령어의 경우 메모리에서 데이터를 읽고, store명령어의 경우 메모리에 데이터를 쓴다

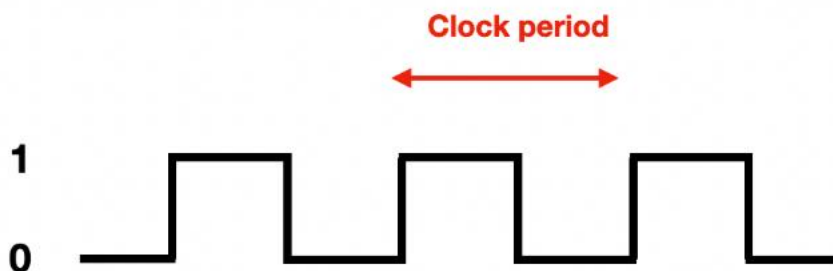
5: Write Back:

- 연산 결과를 레지스터 파일에 저장한다.

● Control logic이란

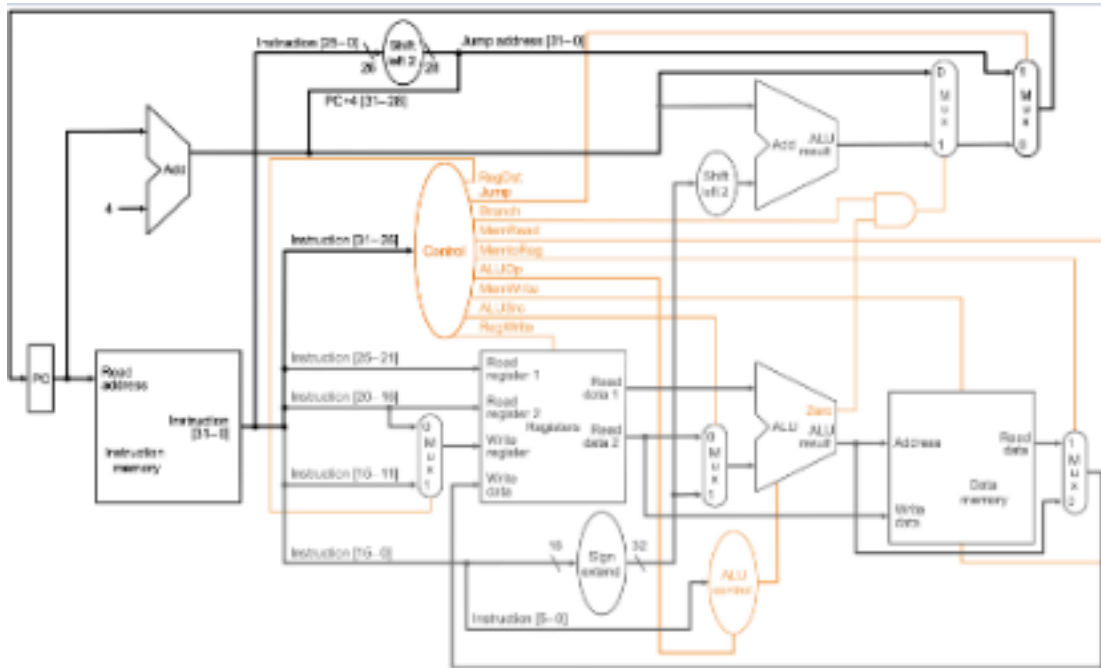
Control logic은 각 구성 요소의 동작을 조정하는 중요한 요소이다. Control logic은 명령어를 해석하여 control signal을 생성하고, 이 신호들은 프로세서의 다양한 하드웨어 유닛의 동작을 지시한다.

● Single Cycle이란



CPU는 내부 회로를 동작시키기 위해 일정한 주기로 규칙적인 전기 신호를 발생시키는데 이를 Clock이라고 한다. 즉 1 clock cycle은 한 번의 전기 신호를 말하고, 한 클럭에 걸리는 시간을 클럭 주기(Clock Period 또는 Clock cycle time)이라고 한다. 싱글 사이클 프로세서는 그 사이클 동안 하나의 명령어를 처리하는 프로세서를 의미한다.

2) 디자인



코드의 디자인은 data path 와 같이 명령어 fetch, decode, execute, memory access, write back 이라는 파이프라인 단계를 중심으로 이루어져 있다. 각 단계는 함수로 구현되어 있으며 pc 값을 업데이트하는 ALU 가 구현 되어 있다. 명령어와 control unit 은 구조체로 되어 있다.

3) 코드설명

```
// 빅 엔디안 변환 함수 ▶
unsigned int convertToBigEndian(unsigned int value) {
    unsigned char* bytes = (unsigned char*)&value;
    return ((unsigned int)bytes[3]) | ((unsigned int)bytes[2] << 8) |
           ((unsigned int)bytes[1] << 16) | ((unsigned int)bytes[0] << 24);
}
```

빅 엔디안 변환 함수: 이 함수는 리틀 엔디안 형식으로 저장된 값을 빅 엔디안으로 변환한다. 데이터를 올바르게 해석하기 위함이다.

```
// 부호 확장 함수
int SignExtend(short value) {
    return (int)value;
}
```

부호 확장 함수: 이 함수는 16비트의 imm값을 32비트로 확장시켜 반환한다.

```
// 명령어
typedef struct {
    int opcode;
    int rs;
    int rt;
    int rd;
    int shamt;
    int funct;
    int constant;
    int address;
} inst;
```

Inst 구조체: 명령어의 각 필드를 저장한다.

```

//control signal
void CU_signal(control* CU, Inst* decodedInst, char type) {
    CU->RegDst = 0;
    CU->RegWrite = 0;
    CU->ALUSrc = 0;
    CU->ALUOp = 0;
    CU->MemRead = 0;
    CU->MemWrite = 0;
    CU->MemtoReg = 0;
    CU->PCSrc = 0;

    if (type == 'R') {
        CU->RegDst = 1;
        CU->RegWrite = 1;
        CU->ALUSrc = 0;
        CU->ALUOp = 2; // R형 명령어 ALUOp 설정
    }
    else if (type == 'I') {
        CU->RegWrite = 1;
        CU->ALUSrc = 1;
        switch (decodedInst->opcode) {
            case 0x23: // lw
            case 0x20: // lb
            case 0x24: // lbu
            case 0x21: // lh
            case 0x26: // lhu
                CU->MemRead = 1;
                CU->MemtoReg = 1;
                CU->ALUOp = 0; // ADD
                break;
            case 0x2b: // sw
                CU->MemWrite = 1;
                CU->ALUOp = 0; // ADD
                break;
            case 0x04: // beq
                CU->PCSrc = 1;
                CU->ALUOp = 1; // SUB
                break;
            case 0x08: // addi
            case 0x09: // addiu
                CU->ALUOp = 0; // ADD
                break;
            case 0x0c: // andi
                CU->ALUOp = 3; // AND
                break;
            case 0x0d: // ori
                CU->ALUOp = 4; // OR
                break;
            case 0x0a: // slli
            case 0x0b: // sitlu
                CU->ALUOp = 6; // SLT
                break;
        }
    }
    else if (type == 'J') {
        // J형 명령어는 ALU 연산을 사용하지 않으므로 설정하지 않음
    }
}

```

CU_signal: 명령어 type에 따라 control signal을 설정한다.

```

int Adder(int pc, int input) {
    return pc + (input << 2); // 4바이트 단위로 증가시키기
}

int branchAdder(int pc, int constant) {
    return pc + 4 + (constant << 2);
}

int jumpAdder(int pc, int address) {
    return ((pc + 4) & 0xf0000000 | (address << 2));
}

```

PC 및 ALU: pc는 현재 실행 중인 명령어의 주소를 가리킨다. 그리고 명령어 실행 후에 업데이트 된다. 또한, brach, jump의 PC값 업데이트를 위한 ALU를 구현하였다.

```

//memory access
void memoryAccess(inst* decodedInst, control* CU) {
    printf(" [Memory Access] ");
    unsigned int address;
    unsigned int value;

    switch (decodedInst->opcode) {
        case 0x23: // lw
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = memory[address / 4];
            printf("Load, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        case 0x20: // lb
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = memory[address / 4] & 0xff; // 8비트만 가져옴
            printf("Load, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        case 0x24: // lbu
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = memory[address / 4] & 0xff; // 8비트만 가져옴
            printf("Load, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        case 0x21: // lh
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = memory[address / 4] & 0xffff; // 16비트만 가져옴
            printf("Load, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        case 0x25: // lhu
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = memory[address / 4] & 0xffff; // 16비트만 가져옴
            printf("Load, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        case 0x2b: // sw
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = Reg[decodedInst->rt];
            memory[address / 4] = value;
            printf("Store, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        case 0x2d: // sb
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = Reg[decodedInst->rt] & 0xff; // 8비트만 저장
            memory[address / 4] = (memory[address / 4] & 0xffffffff00) | value;
            printf("Store, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        case 0x29: // sh
            address = Reg[decodedInst->rs] + decodedInst->constant;
            value = Reg[decodedInst->rt] & 0xffff; // 16비트만 저장
            memory[address / 4] = (memory[address / 4] & 0xffffffff0000) | value;
            printf("Store, Address: 0x%05x, Value: %02x\n", address, value);
            break;
        default:
            printf("Pass\n");
            break;
    }
}

```

Memory access: 메모리 접근 명령어에 대해 각 명령어에 따라서 메모리에서 데이터를 읽거나 쓰는 작업을 수행한다.


```

int execute(inst* decodedInst) {
    int validALUresult = 0; // ALUresult 유효성 확인 변수

    if (type == 'R') {
        switch (decodedInst->func) {
            case 0x20: // ADD
                func = "add";
                ALUresult = Reg[decodedInst->rs] + Reg[decodedInst->rt];
                Reg[decodedInst->rd] = ALUresult;
                validALUresult = 1;
                break;
            case 0x21: // ADDU
                func = "addu";
                ALUresult = (unsigned int)(Reg[decodedInst->rs] + Reg[decodedInst->rt]);
                Reg[decodedInst->rd] = ALUresult;
                validALUresult = 1;
                break;
        }
    }
}

```

```

//writeback
void writeback(inst* decodedInst) {
    switch (type) {
        case 'R':
            printf("    [Write Back] Target: %s, Value: 0xX08x / newPC: 0xX08x\n", register_names[decodedInst->rd], Reg[decodedInst->rd], pc);
            break;
        case 'I':
            printf("    [Write Back] Target: %s, Value: 0xX08x / newPC: 0xX08x\n", register_names[decodedInst->rt], Reg[decodedInst->rt], pc);
            break;
        case 'J':
            printf("    [Write Back] / newPC: 0xX08x\n", pc);
            break;
        default:
            printf("    [Write Back] / newPC: 0xX08x\n", pc);
            break;
    }
}

```

Writeback: execute 함수를 실행하고 명령어의 최종 결과를 레지스터 파일이나 메모리에 저장하는 writeback을 실행하고 writeback함수에서 각 명령어에 맞는 값을 출력한다.

4) 구현

Implemented parts	o/x
fetch	O
decode	O
execute	O
Memory access	O
Write back	O
MUX	X

- MUX의 구현실패

본 프로그램에서 MUX를 구현하는 데 어려움을 겪었다. MUX는 cpu 설계에서 여러 입력 중 하나를 선택하여 출력으로 보내는 중요한 역할을 한다. 그러나 MUX는 여러 신호를 제어하기 위해 복잡한 로직이 필요하다. 각 입력 신호가 올바른 조건에서 선택되어야 하며, 이 조건들을 정확히 구현하는 데 어려움이 있었다.

- Write back구현

Write back 단계를 별도의 함수로 분리하여 구현하고자 하였으나 파이프라인의 각 단계가 밀접하게 연결되어 있어, Write back단계를 분리하는데 실패하였습니다.

5) 프로그램 실행

KOI에서 프로그램실행 방법

- ① 터미널을 열고 `gcc -o 32217072 32217072.c` 커맨드를 작성하여 컴파일 한다.
- ② bin파일을 준비한다. 파일이름은 test로 가정한다.
- ③ `./32217072 test.bin` 커맨드를 작성한다.

KOI에서 프로그램 실행 출력

다음 이미지는 HW2 input1 ~ 5를 순서대로 사용

1.sum.bin

```
32217072> cycle: 145
[Instruction Fetch] 0x27bd0010 (pc=0x0000060)
[Instruction Decode] Type: I inst: addiu $sp, $sp, 16
      Opcode: 9 RS: 29 (0x1000000) RT: 29 (0x1000000) Imm: 16
      RegDst: 0 RegWrite: 1 ALUSrc: 1 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 0
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $sp, Value: 0x01000000 / newPC: 0x0000064
32217072> cycle: 146
[Instruction Fetch] 0x03e00008 (pc=0x0000064)
[Instruction Decode] Type: R inst: jr $zero $ra $zero
      Opcode: 0 RS: 31 (0xffffffff) RT: 0 (0x0) RD: 0 (0x0) Shamt: 0 Funct: 8
      RegDst: 1 RegWrite: 1 ALUSrc: 0 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 2
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $zero, Value: 0x00000000 / newPC: 0xffffffff
32217072> Final Result
Cycles: 146, R-type instructions: 45, I-type instructions: 101, J-type instructions: 0
Return value (v0): 45
koicloud@ca-32217072:~/vscode$
```

2.func.bin

```
32217072> cycle: 94
[Instruction Fetch] 0x27bd0028 (pc=0x0000030)
[Instruction Decode] Type: I inst: addiu $sp, $sp, 40
      Opcode: 9 RS: 29 (0x1000000) RT: 29 (0x1000000) Imm: 40
      RegDst: 0 RegWrite: 1 ALUSrc: 1 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 0
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $sp, Value: 0x01000000 / newPC: 0x0000034
32217072> cycle: 95
[Instruction Fetch] 0x03e00008 (pc=0x0000034)
[Instruction Decode] Type: R inst: jr $zero $ra $zero
      Opcode: 0 RS: 31 (0xffffffff) RT: 0 (0x0) RD: 0 (0x0) Shamt: 0 Funct: 8
      RegDst: 1 RegWrite: 1 ALUSrc: 0 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 2
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $zero, Value: 0x00000000 / newPC: 0xffffffff
32217072> Final Result
Cycles: 95, R-type instructions: 31, I-type instructions: 60, J-type instructions: 4
Return value (v0): 10
koicloud@ca-32217072:~/vscode$
```

3.fibonacci.bin

```
32217072> cycle: 46371
[Instruction Fetch] 0x27bd0020 (pc=0x0000028)
[Instruction Decode] Type: I inst: addiu $sp, $sp, 32
    Opcode: 9 RS: 29 (0x1000000) RT: 29 (0x1000000) Imm: 32
    RegDst: 0 RegWrite: 1 ALUSrc: 1 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 0
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $sp, Value: 0x01000000 / newPC: 0x000002c
32217072> cycle: 46372
[Instruction Fetch] 0x03e00008 (pc=0x000002c)
[Instruction Decode] Type: R inst: jr $zero $ra $zero
    Opcode: 0 RS: 31 (0xffffffff) RT: 0 (0x0) RD: 0 (0x0) Shamt: 0 Funct: 8
    RegDst: 1 RegWrite: 1 ALUSrc: 0 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 2
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $zero, Value: 0x00000000 / newPC: 0xffffffff
32217072> Final Result
    Cycles: 46372, R-type instructions: 14798, I-type instructions: 29601, J-type instructions: 1973
    Return value (v0): 610
koicloud@ca-32217072:~/vscode$
```

4.factorial.bin

```
32217072> cycle: 138
[Instruction Fetch] 0x27bd0020 (pc=0x0000028)
[Instruction Decode] Type: I inst: addiu $sp, $sp, 32
    Opcode: 9 RS: 29 (0x1000000) RT: 29 (0x1000000) Imm: 32
    RegDst: 0 RegWrite: 1 ALUSrc: 1 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 0
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $sp, Value: 0x01000000 / newPC: 0x000002c
32217072> cycle: 139
[Instruction Fetch] 0x03e00008 (pc=0x000002c)
[Instruction Decode] Type: R inst: jr $zero $ra $zero
    Opcode: 0 RS: 31 (0xffffffff) RT: 0 (0x0) RD: 0 (0x0) Shamt: 0 Funct: 8
    RegDst: 1 RegWrite: 1 ALUSrc: 0 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 2
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $zero, Value: 0x00000000 / newPC: 0xffffffff
32217072> Final Result
    Cycles: 139, R-type instructions: 53, I-type instructions: 81, J-type instructions: 5
    Return value (v0): 5
koicloud@ca-32217072:~/vscode$
```

5.power.bin

```
32217072> cycle: 103
[Instruction Fetch] 0x27bd0020 (pc=0x000002c)
[Instruction Decode] Type: I inst: addiu $sp, $sp, 32
    Opcode: 9 RS: 29 (0x1000000) RT: 29 (0x1000000) Imm: 32
    RegDst: 0 RegWrite: 1 ALUSrc: 1 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 0
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $sp, Value: 0x01000000 / newPC: 0x0000030
32217072> cycle: 104
[Instruction Fetch] 0x03e00008 (pc=0x0000030)
[Instruction Decode] Type: R inst: jr $zero $ra $zero
    Opcode: 0 RS: 31 (0xffffffff) RT: 0 (0x0) RD: 0 (0x0) Shamt: 0 Funct: 8
    RegDst: 1 RegWrite: 1 ALUSrc: 0 PCSrc: 0 MemWrite: 0 MemtoReg: 0 ALUOp: 2
[Execute] ALU result: 01000000
[Memory Access] Pass
[Write Back] Target: $zero, Value: 0x00000000 / newPC: 0xffffffff
32217072> Final Result
    Cycles: 104, R-type instructions: 38, I-type instructions: 62, J-type instructions: 4
    Return value (v0): 10
koicloud@ca-32217072:~/vscode$
```

3. 결론

Single cycle 시뮬레이터를 구현하면서 data path를 보고 이것을 코드로 구현할지에 대해 생각을 많이 해보았다. 무작정 코딩하는 것 보다 개념을 확실히 알고 설계를 한 뒤 코드를 작성하는 것이 효율적이라는 것을 느꼈다.

명령어를 가져오고 디코딩할 때 빅 엔디안으로 변환해야 한다는 생각을 떠올리는 것이 어려웠고, branch, jump 명령어들의 pc값 update를 할 때 고민을 가장 많이 했던 것 같다. 또한 코드의 각 부분이 제대로 작동하는지 확인하는 과정에서 많은 디버깅과 테스트가 필요했다. 특히, 복잡한 명령어 세트를 다루다 보니, 각 명령어가 올바르게 실행되는지 확인하는 것이 중요했고, 다양한 테스트 프로그램의 출력의 오류를 수정하는 과정을 반복했다.

아쉬운 점은 control signal이 실제 data path처럼 구현되게 하지 못했다는 점이고, 또 MUX를 구현하지 못한 점이 아쉬움으로 남는다. 하지만 single cycle과제를 하면서 data path가 실제로 어떻게 진행되는지 더 깊게 알 수 있게 되었다.